

Optymalizacja Kodu Na Różne Architektury

Zadanie 1

Mateusz Ryś

1. Procesor parametry

Parametr	Wartość
Producent	AMD
Model	Ryzen 5 5600H
Mikroarchitektura	Zen 3
Rdzenie/Wątki	6/12
Częstotliwość bazowa	3.3 GHz
Częstotliwość turbo	4.2 GHz
Cache	16MB (L3) + 3MB (L2)
GFLOPS	316.8
GFLOPS/rdzeń	52.8

Architektura Zen 3 dysponuje jednostkami AVX2 o szerokości 256 bitów oraz wsparciem dla instrukcji FMA (Fused Multiply-Add)

2. Optymalizacje

2.1 Lista optymalizacji

- Prealokacja pamięci
- In-place transformation
- Zastosowanie iteratorów i algorytmów standardowych
- Wersja bare-metal (czyste wskaźniki)
- Wersja równoległa

2.2 Dostosowanie do procesora

Kluczowym etapem optymalizacji jest kompilowanie programu z flagą `-march=znver3`. Pozwala kompilatorowi używać specyficznych instrukcji jakie używa dana architektura (tu Zen3), w tym pełne wykorzystanie możliwości jednostek AVX2. Również warto zastanowić się nad flagą `-O2`.

3. Opis implementacji

3.1 Wersja bazowa

- Standardowe użycie obiektów typu `std::string` oraz jej metod.
- Funkcja normalizująca została podzielona na 3 funkcje
 - główną, która zastępuje pojedyncze znaki
 - funkcje usuwającą wielokrotne spacje
 - oraz funkcje usuwającą duplikaty
- Zatem 3 razy przechodziły po całym tekście
- Do wyluskania słów użyto `stringstream`

Zaznaczę jeszcze, że pisałem usuwanie duplikatów po napisaniu dwóch pozostałych funkcji. Wtedy nie myślałem czego użyję do wyeliminowania ich. Postanowiłem jednak pozostawić osobne usuwanie spacji nadmiarowych by w większej ilości miejsc móc zastosować techniki optymalizacyjne.

3.2 Prelokacja pamięci

Tu nie dużo zostało zmienione. Tylko przy definiowaniu zmiennej typu string od razu przypisujemy mu tyle pamięci ile może maksymalnie wynosić tekst wynikowy (metoda `reserve()`).

3.3 In-place transformation

Zamiast zwracać nowy obiekt string (przy okazji tworząc nowy), zmiany są wykonywane na podanej zmiennej (zmienna string przekazywana poprzez referencje). Weszły tu operacje wykonywane na dwóch wskaźnikach (zapisu i czytania) oraz metoda `resize` do zmiany rozmiaru zmiennej. Nie została ta optymalizacja zastosowana w kontekście metody usuwającej duplikaty z powodu trudności jakie by się wiązały z wprowadzeniem tych zmian.

3.4 Zastosowanie iteratorów

Tutaj zastępujemy zwykłe pętle, pętlami wykorzystującymi Iteratory. W funkcjach `normalize` oraz `remove_extra_spaces` użyto funkcji `transform`, `remove_if`, `unique` oraz `erase`. W `delete_duplicates` została przepisana. Teraz to używamy pointerów na początek i koniec słowa i wykorzystując metody typu `equal` prosto porównujemy słowa wskazane przez iteratory.

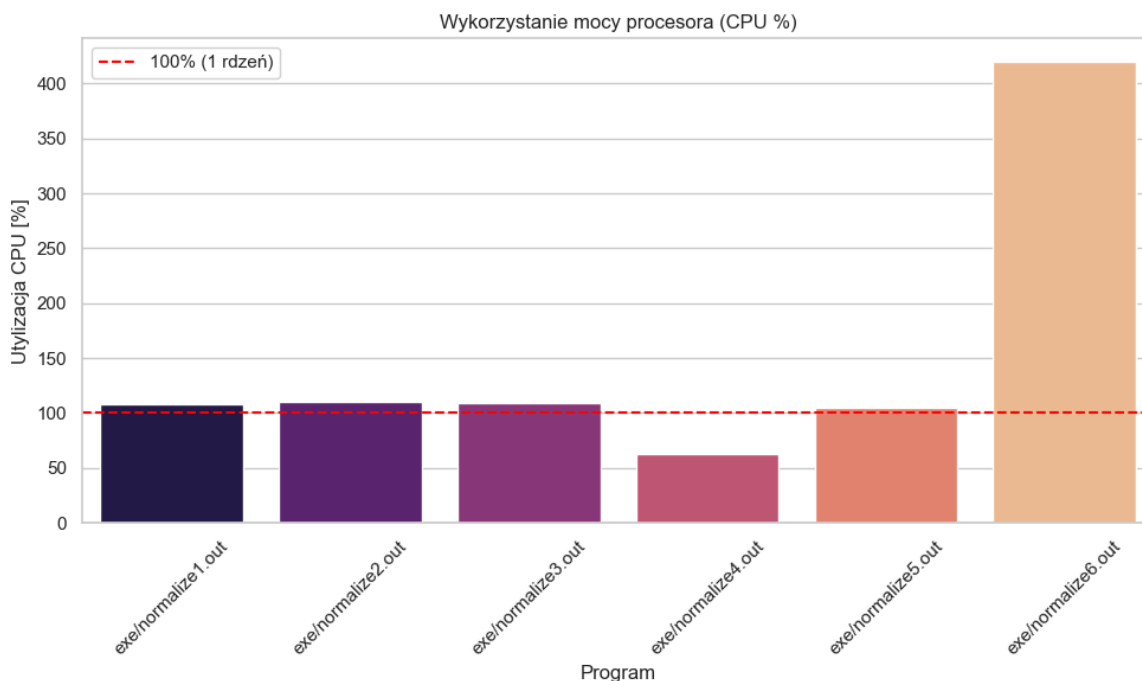
3.5 Bare metal

W tym rozwiązaniu przechodzimy na tablicze typu `char` oraz użycie wskaźników. Bardzo podobne do iteratorów ale trzeba pamiętać o zwalnianiu powstałych bloków pamięci

3.6 Rozwiązanie współbieżne

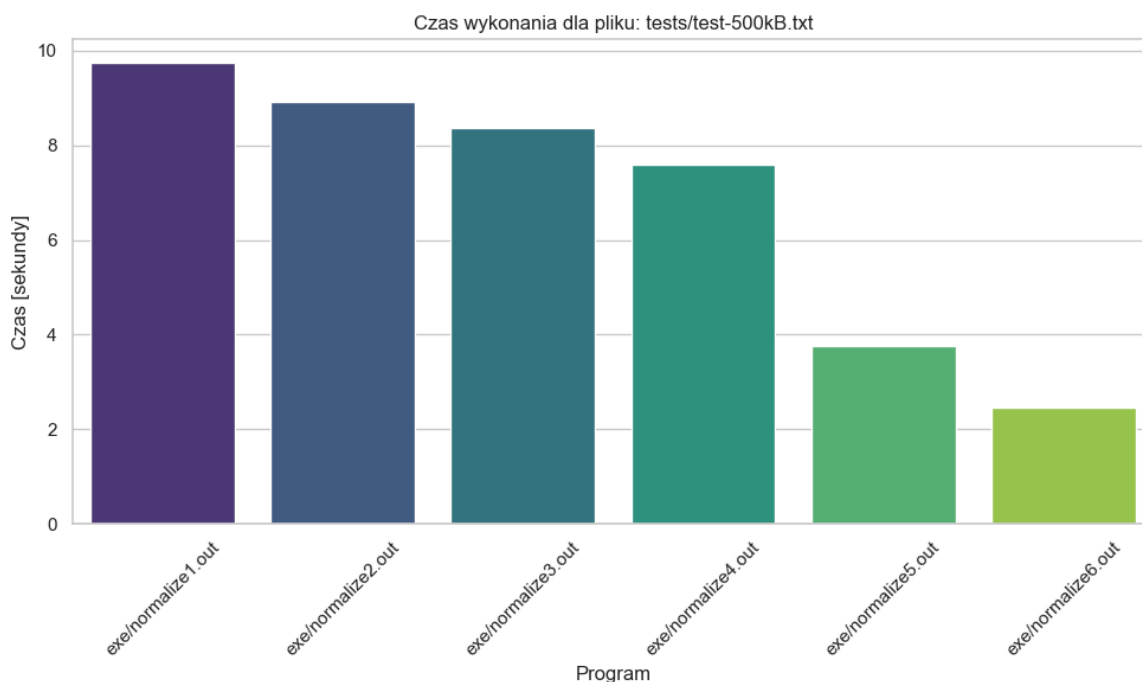
Jedyna wersja gdzie samych funkcji nie zmieniamy ale zmianie poddajemy `main`'a. Tekst podzieliłem na 4 mniejsze części i na nich wykonuję funkcję `normalize` w osobnych wątkach. Na koniec łączę wynik i jeszcze raz przechodzę po całości funkcją `delete_duplicates` w celu usunięcia duplikatów na granicy. Trzeba pamiętać, że przy kompilacji należy dodać flagę `-fopenmp`, ponieważ korzystam z biblioteki `omp` do wielowątkowości.

4 Wyniki (dla największego z plików) i wnioski



Wykorzystanie CPU

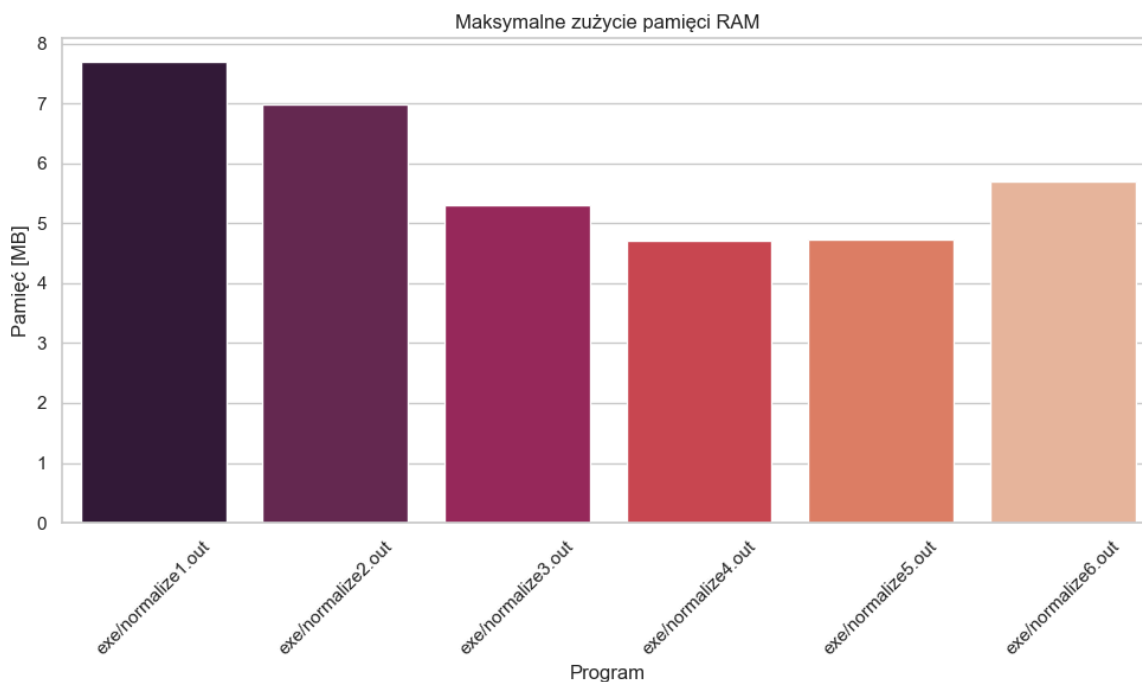
Wartość dla ostatniego programu pokazuje nam, że rzeczywiście mamy większe wykorzystanie procesora do wykonania normalizacji, co idzie w parze w uzyskaniu wyniku w szybkim czasie. Tzn algorytm nie jest ograniczony do jednego rdzenia i działa na wielu wątkach. Za to większość programów oscyluje wokół wartości 100% co wskazuje na pracę w obrębie jednego wątku.



Wykorzystanie czasu

Z wykresu wynika, że kolejne wersje powodowały krótsze czasy oczekiwania na wyniki. Najlepsze czasy dają bare metal oraz wersja równoległa która bazuje na niskopoziomowej

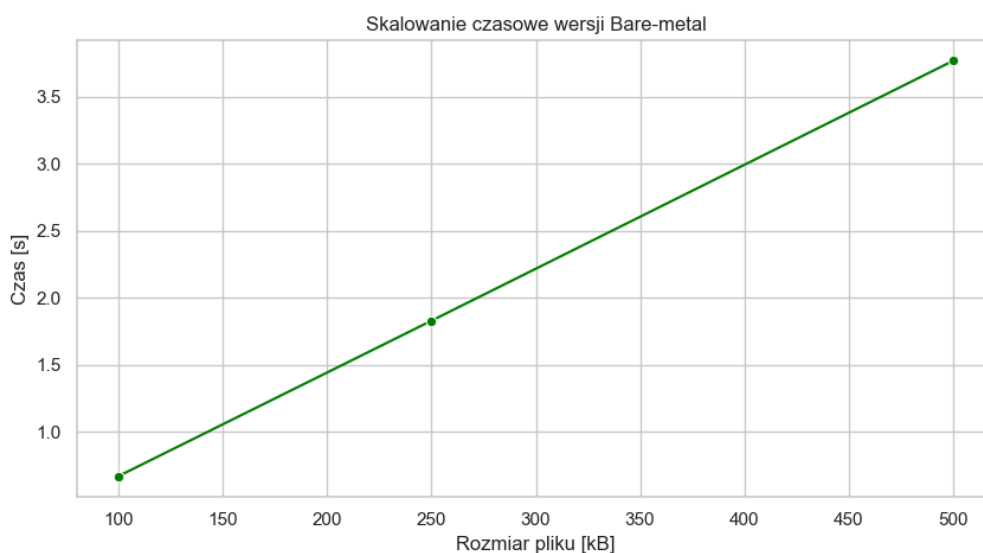
implementacji. Z tego można wnioskować, że rozwiązania wysokopoziomowe idą w parze z narzutem który powoduje sporo dłuższy czas oczekiwania na wyniki.



Wykorzystanie RAM

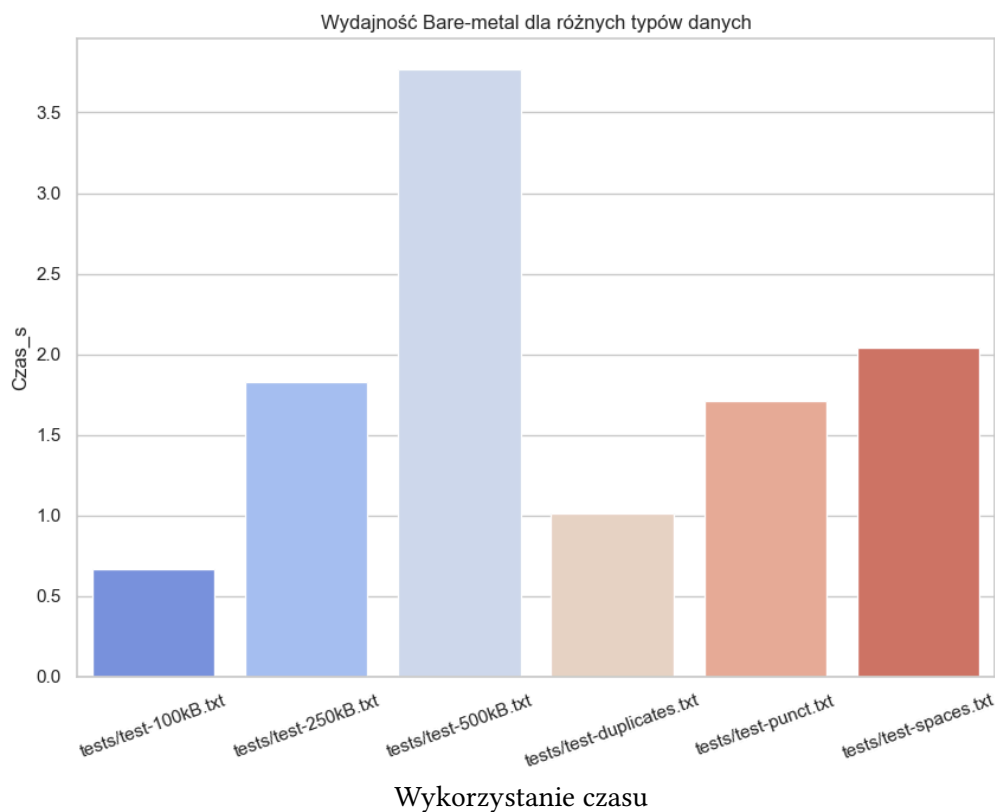
Pierwsze 3 optymalizacje zmniejszają zużycie RAMu. Pozwoliło to zredukować znacząco obciążenie. Za to użycie OpenMP spowodowało zwiększenie tego zapotrzebowania. Zatem jak chcemy przyspieszyć program owymi mechanikami musimy liczyć się z tym skokiem.

Zobaczmy zatem jak szybsza z wersji (bare metal) radzi sobie z różnymi danymi.



Wykorzystanie czasu

Można tu zaobserwować fakt, że rośnie nam czas liniowo. Zatem mamy naoczny dowód, że nasz algorytm ma złożoność $O(n)$.



Tu mamy czasy dla większych plików oraz kolejno teksty które:

- mają większość duplikatów
- mają większość interpunkcji
- mają większość znaków białych

Jak widzimy najgorzej program radzi sobie ze znakami białymi.

Podsumowanie

Projekt pozwolił na zaobserwowanie jaki wpływ na program mają poszczególne optymalizacje. Przejście z modelu wysokopoziomowego na niskopoziomowy (bare-metal) oraz wdrożenie zrównoleglenia pozwoliło na duże przyspieszenie programu. Również zużycie ramu zostało racjonalnie zmniejszone.